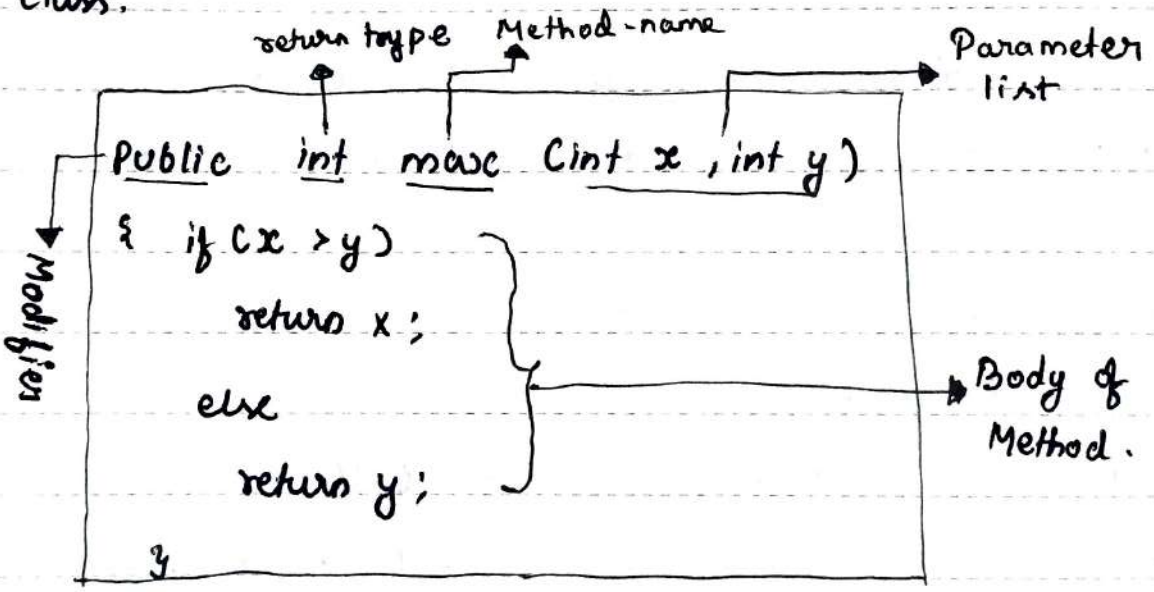
Java Methods :-

*) Java Methods are blocks of code that perform a specific task.

*) A method allows to write a piece of logic once and reuse it wherever needed.

*) Helps to keep our code clean, organized easier to understand and manage.

*) All methods in java must belong to class.



Why use Methods ?

*) Code Reusability

*) Modularity

*) Readability

*) Maintainability

Method call stack in Java :-

*) Java is an object-oriented and stack based programming language.

*) When method is called, Java uses internal structure known as call stack to manage execution, variables and return addresses.

What is call stack ?

*) Call stack is data structure used by program during runtime to manage calls & local variables.

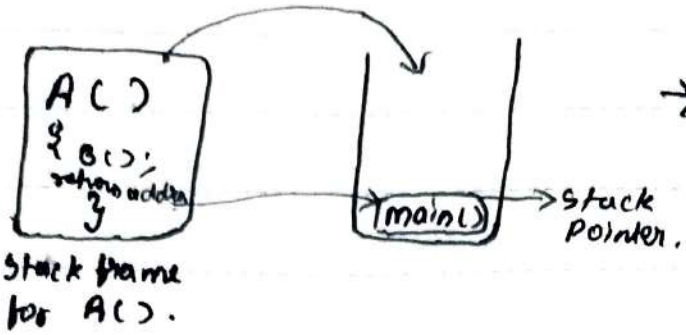
*) It operates in (LIFO) Last In First Out

How Methods are executed ?

When a method is called:

- A new stack frame is add to call stack
- The frame stores the method details in stack
- The method runs its code
- After execution, stack frame is removed
- Control goes back to function which called this method.

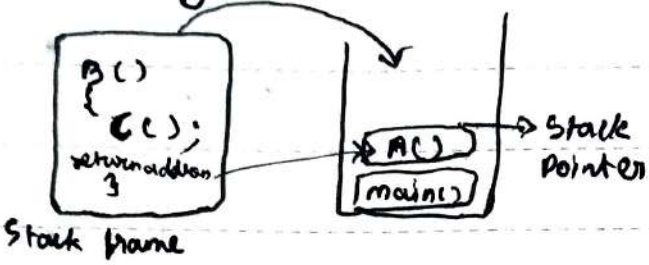
JVM automatically manages the call stack



→ Stack frame for A() is created and pushed into call stack

→ return address will be saved for A()

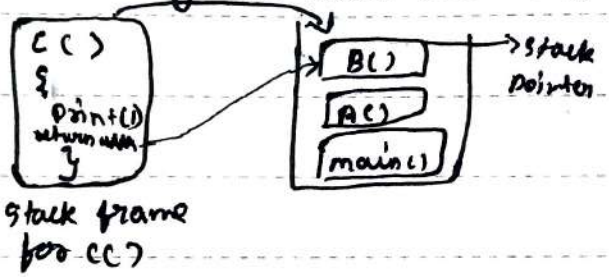
Let's say A() calls the B():



→ As B() is called by A(), stack frame of B() will be created and pushed to call stack.

→ return address will be saved for B()

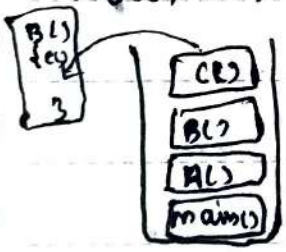
Let's say B() calls the C():



→ Same, as B() called C(). Stack frame of C() is created and pushed to call stack.

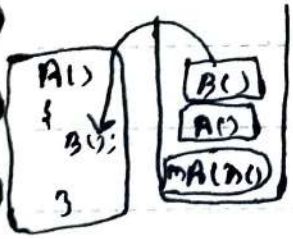
→ return address will be saved for C().

Now, C() will execute and print().



→ After C() executes, it returns to B() function and removed from call stack.

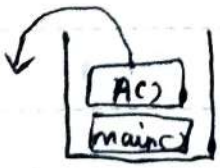
→ It returns to B() using return address.



→ After B() executes the remaining code and returns to A() and removed from call stack

→ B() returns to A() using return address.

Then A() executes and return to main() and removed using return address



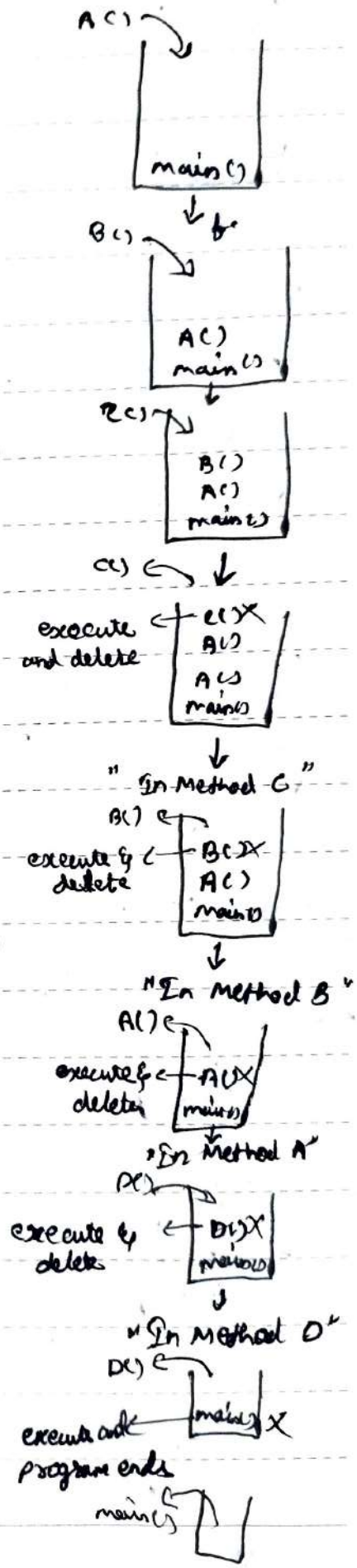
Now control comes to main() using return address

After main() returns 0, stack frame of main also deleted

```
Public class CallStackExample {

    Public static void D()
    {
        float d = 40.5f;
        Sout ("In Method D");
    }

    Public static void C()
    {
        Sout ("In Method C");
    }

    Public static void B()
    {
        C(); // calling C()
        Sout ("In Method B");
    }
    Public static void A()
    {
        B(); // calling B()
        Sout ("In Method A");
    }

    public static void main(String[] args)
    {
        A(); // start with function A
        D(); // Then call D
    }
}
```

O/P:-

In Method C
In Method B
In Method A
In Method D



Types of Methods in Java :-

1. Predefined Method.
   - → Already defined in Java libraries
   - → Ex: random() method which is present in "Math" class and we can call it using the "ClassName.methodName()"

```
Math.random()   // returns random value
Math.PI         // returns vals of π as a constant.
                   it is a constant not a method.
```

2. Userdefi~~ned~~ned Method:
   - → Written by user. Modified according to requirement.

```
→ int Hello()
  {  Sout("Hi");
  }
```

Different ways to create Java Method :-

There are 2 ways to create a method in java:

1. Instance Method
2. Static Method

Instance Methods in Java :-

*) An instance method belongs to an object of a class.

*) To run this method, it requires object of class.

*) You can't run it or call it without help of object of class.

*) It can access and modify the Object's instance variables.

*) It can also call other instance or static method.

```
class Person {
    String name = "Alice";
    // Instance method
    void greet()
     { sout ("Hello" + name + "!");
     }
    public static void main(String[] args)
     { //greet(); // error, you can't call without object.
     Person p = new Person(); // create object
      p.greet(); // call with help of object.
 }  }
```

2 types of instance Methods :-

*) Getters → Used to retrieve value of Private instance variable

*) Setters → Used to modify value of Private instance variable

```
class Account {
    Private int balance =100;

    Public int getBalance () // getter func
    { return balance;
    }

    Public void setBalance (int amt) // setter func
    { balance += amt;
    }

    Public static void main (String[] args )
    { Account acc = new Account();
    sout(acc. getBalance());  //100 (only able to access
                                 can't modify)

      acc. setBalance (50);  // can able to modify
                                the variable
    sout(acc. getBalance ()); //150
    }
}
```

Static Methods in Java :-

*) Static keyword is used to create a method that belongs to class, not specific to any objects like Instance Method.

*) It can be accessed without any object help. also

*) Can be accesed by all instance of class, but does not rely on any specific instance.

*) Static method can access static variable directly without the need for an object.

*) They cannot access non-static variables (instance) or methods directly.

*) Static methods can be accessed directly in both static and non-static contexts.

*) "this" and "super" are not allowed to be used in static environment.

Things to know :-

```
Public static void main (String[] args)
{

}
```

is also static, so that only it runs without any object help.

```
Public class Geeks {
    Static int a = 40;
    int b = 30;
    void display ()
    { sout (a); // static variable can be accessed in non
                                    static method.
      sout (b);
    }

    static void staticDisplay ()
    { sout (a);
      //sout(b); // throws error non-static variable
                        can't be accessed inside static method.
      this.a; // error.
    }

    public void main (String[] args )
    {
        sout ( staticDisplay () ); // static method can be
                                      accessed without object.
        Geeks obj = new Geeks ();
        obj.display ();   // non-static needs obj to access.
        obj. staticDisplay ();   // we can access static
                                    method using obj also.
        Print(); // throws error (if we try to access static method
                                  from another class)
        Sample.print(); // "Hello"
    }
}
                        (to access static method from another
                         class use className.
class Sample {          Syntax; className . Method_Name ())
    Static void print ()
    { sout ("Hello");
    }
}
```